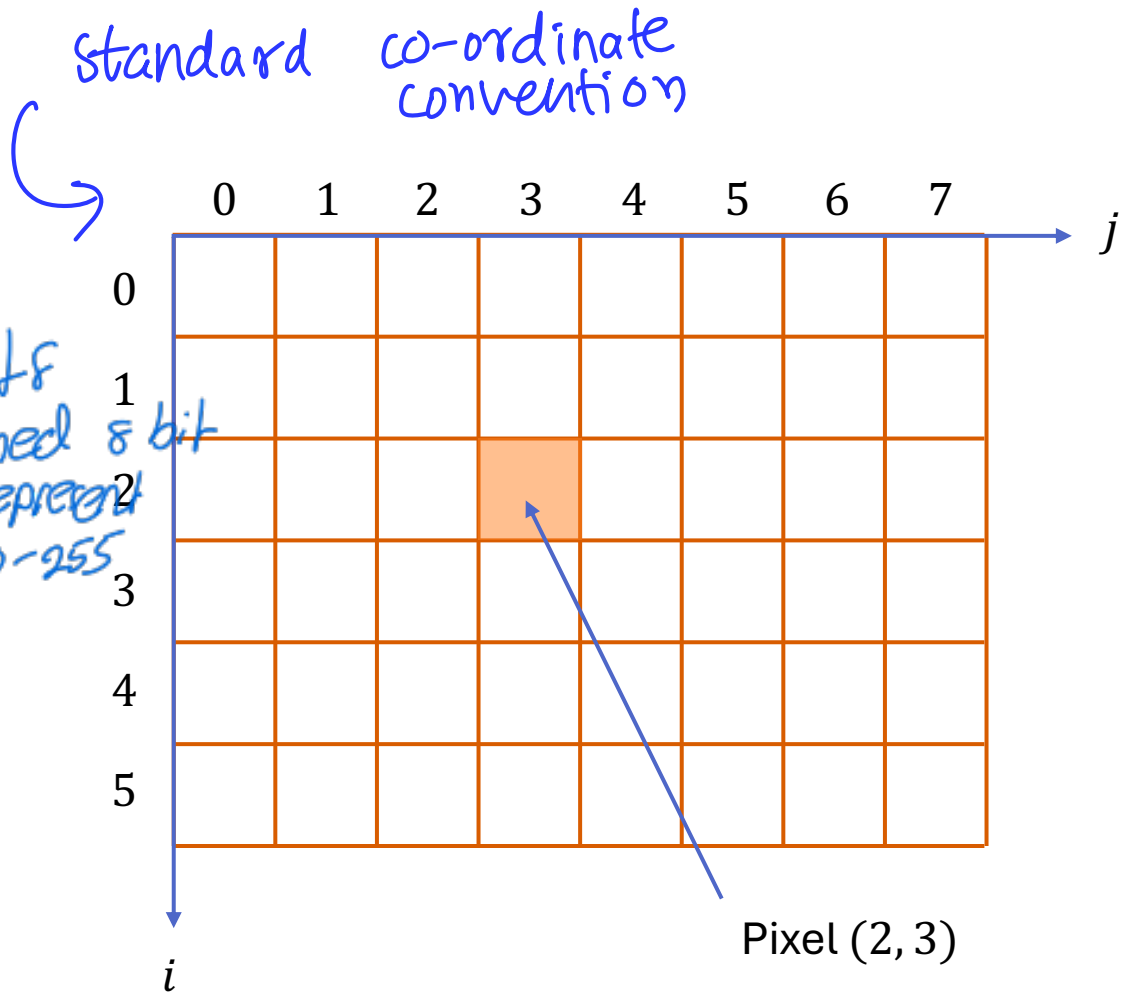
EN3160 L02
Point Operations

What is a Digital Image?

- A grayscale digital image is a rectangular array of numbers which represent pixels.
- Each pixel can take an integer value in the range [0, 255] , for an 8-bit image.
- If the image is a color image, then there would be three such arrays.
- The size of this array is actually the resolution of the image, e.g., example, 3712×5568.
- We take the top-left pixel as the (0, 0) pixel and vertical axis as the i axis.

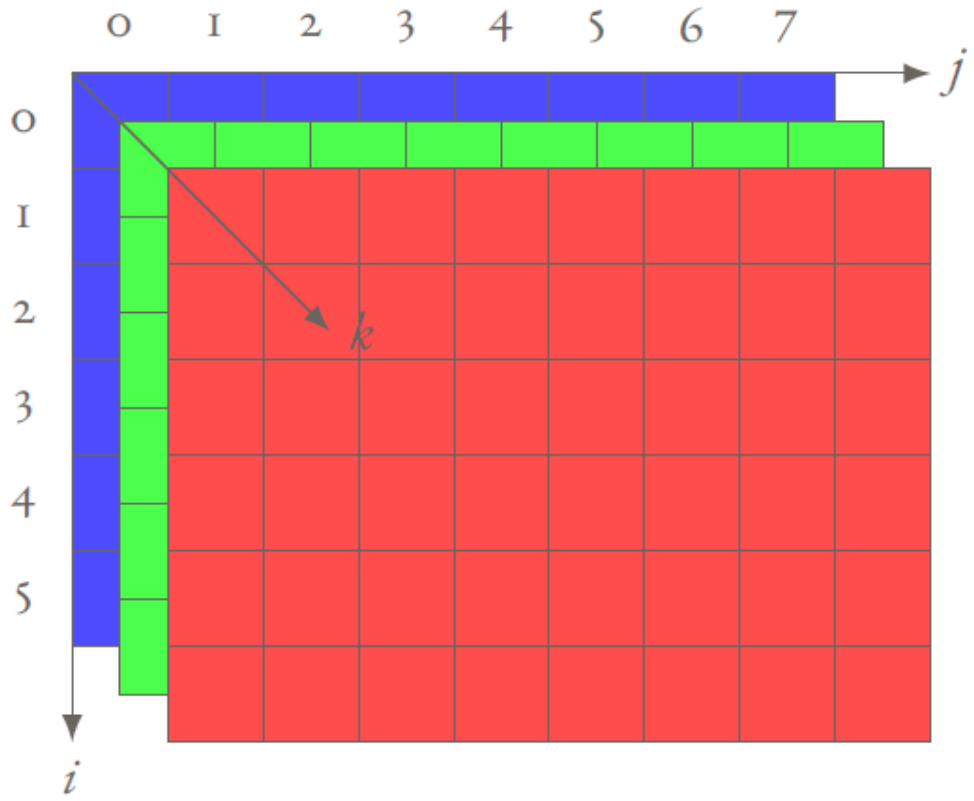


usually pixel values represented in uint8

What is a Color Digital Image?

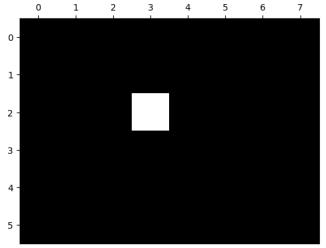
- The grayscale image that we considered as a two-dimensional array, or a single plane.
- A color image has three such planes, one for blue, one for green and one for red. We call such an image an BGR image.
- If we access a pixel location such as (2, 3), we will get three values, B, G, and R.
- Each value is in [0, 255]. In this context, $2^8 \times 2^8 \times 2^8$ different colors are possible.

for a pixel



Creating a 6×8 Image

A Grayscale Image

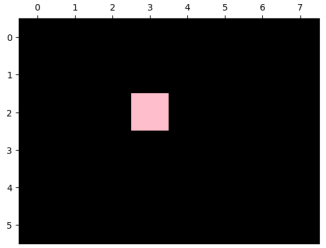


```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
im = np.zeros((6,8), dtype=np.uint8)
im[2,3] = 255
fig, ax = plt.subplots(1, 1, figsize=(6, 8))
ax.imshow(im, cmap='gray', vmin=0, vmax=255)
ax.xaxis.set_ticks_position('top')
plt.show()
```

fully white

we want it to be grayscale otherwise it will map to something

A Color Image



```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
im = np.zeros((6,8,3), dtype=np.uint8)
im[2,3] = (255, 190, 203)
fig, ax = plt.subplots(1, 1, figsize=(6, 8))
ax.imshow(im)
ax.xaxis.set_ticks_position('top')
plt.show()
```

* CV2 uses BGR and matplotlib uses RGB so we need to convert

To work with planes

in [: , : , 0] = 0

B G R

↓

This plane is 0

call x

call y

in B plane is 0

Image Opening and Displaying

Displaying Using Matplotlib

```
import cv2 as cv
import matplotlib.pyplot as plt
im = cv.imread('images/jolie.png')
fig, ax = plt.subplots(1, 1, figsize=(6, 8))
ax.imshow(cv.cvtColor(im, cv.COLOR_BGR2RGB))
ax.xaxis.set_ticks_position('top')
plt.show()
```

converting color

Displaying Using OpenCV

```
import cv2 as cv
im = cv.imread('images/jolie.png')
cv.namedWindow('Image', cv.WINDOW_AUTOSIZE)
cv.imshow('Image', im)
cv.waitKey(0)
cv.destroyAllWindows()
```

Q:Why do we need to cv.cvtColor only when displaying using Matplotlib?

Displaying Image Properties

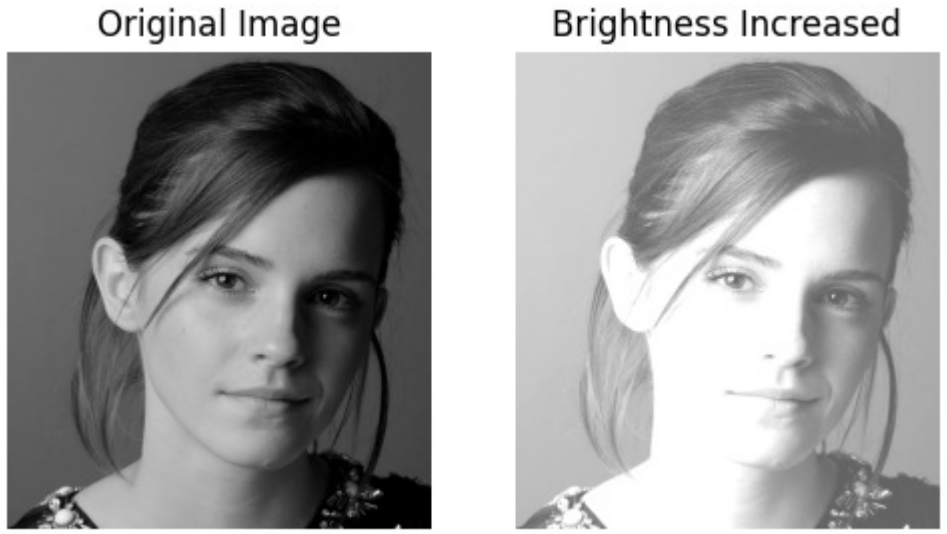
```
import cv2 as cv
import matplotlib.pyplot as plt
im = cv.imread('images/ryan.jpg')
fig, ax = plt.subplots(1, 1, figsize=(6, 8))
ax.imshow(cv.cvtColor(im, cv.COLOR_BGR2RGB))
ax.xaxis.set_ticks_position('top')
plt.show()
print('Image Shape:', im.shape)
print('Image Data Type:', im.dtype)
print('Image Size:', im.size)
```

```
Image Shape: (2641, 1761, 3)
Image Data Type: uint8
Image Size: 13952403
```

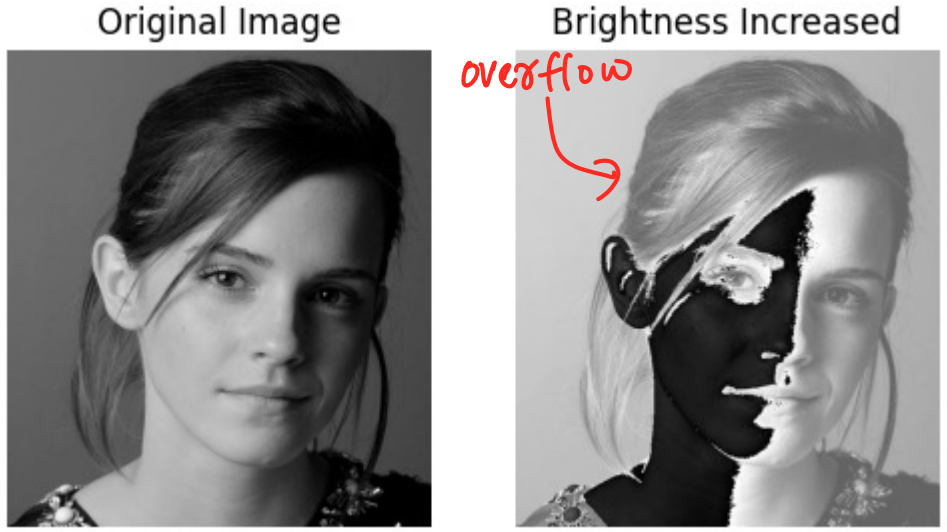
Increasing the Brightness

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
im1 = cv.imread('images/emma_gray.jpg',
cv.IMREAD_GRAYSCALE)
im2 = cv.add(im1, 100)
fig, ax = plt.subplots(1, 2, figsize=(6, 8))
ax[0].imshow(im1, cmap='gray', vmin=0, vmax=255)
ax[0].set_title('Original Image')
ax[1].imshow(im2, cmap='gray', vmin=0, vmax=255)
ax[1].set_title('Brightness Increased')
for a in ax:
    a.axis('off')
plt.show()
```

im2 = im1 + 100 does not work



What is wrong with this? `im2 = im1 + 100`

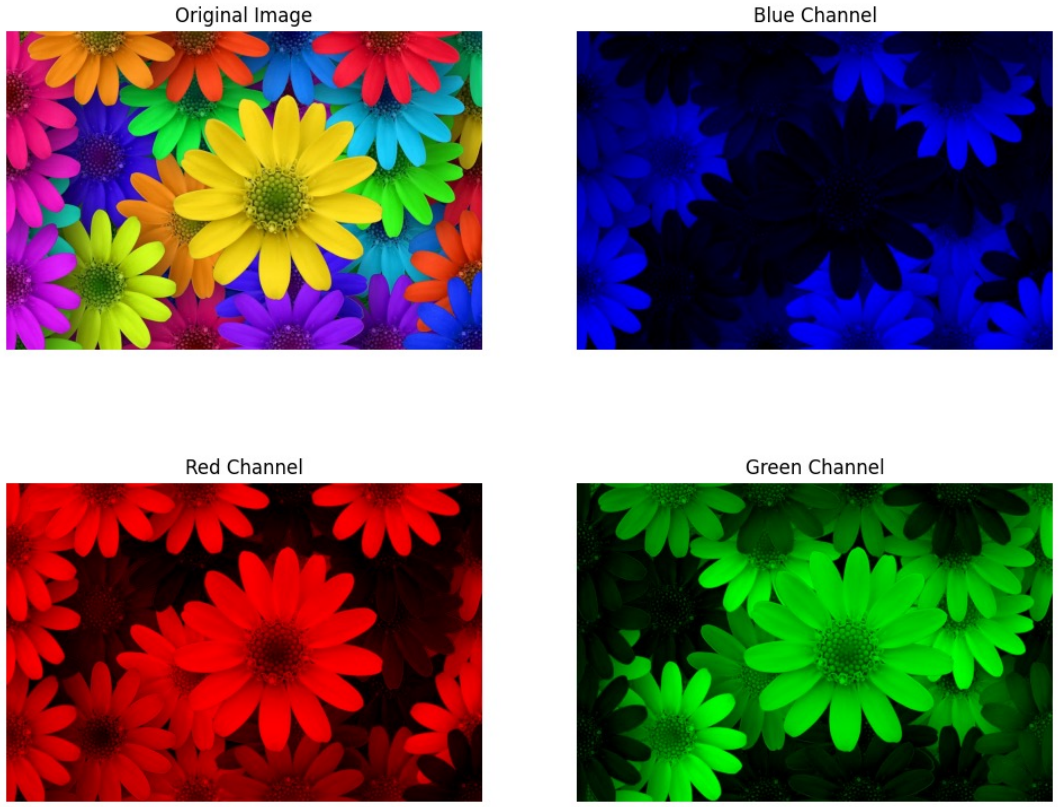


Increasing the Brightness Using Loops (Slow)

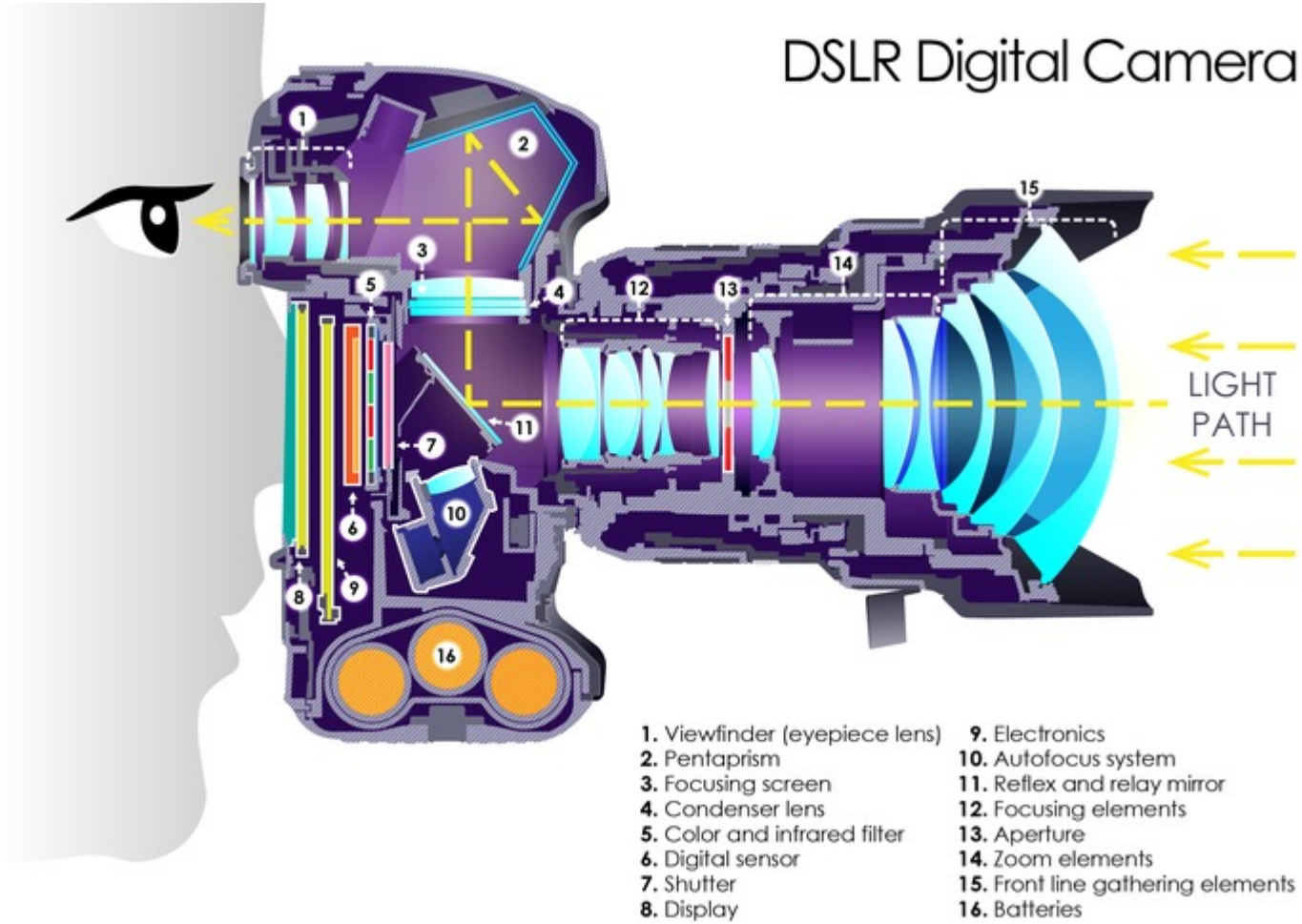
```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
im1 = cv.imread('images/emma_gray.jpg', cv.IMREAD_GRAYSCALE)
im2 = np.zeros_like(im1)
for i in range(im1.shape[0]):
    for j in range(im1.shape[1]):
        im2[i,j] = im1[i,j] + 100
fig, ax = plt.subplots(1, 2, figsize=(6, 8))
ax[0].imshow(im1, cmap='gray', vmin=0, vmax=255)
ax[0].set_title('Original Image')
ax[1].imshow(im2, cmap='gray', vmin=0, vmax=255)
ax[1].set_title('Brightness Increased')
for a in ax:
    a.axis('off')
plt.show()
```

Obtaining One Color Plane

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
im = cv.imread('images/rgb_flowers.jpg')
im_blue = im.copy()
im_blue[:,:,1] = 0
im_blue[:,:,2] = 0
fig, ax = plt.subplots(1, 2, figsize=(12, 8))
ax[0].imshow(cv.cvtColor(im, cv.COLOR_BGR2RGB))
ax[0].set_title('Original Image')
ax[1].imshow(cv.cvtColor(im_blue, cv.COLOR_BGR2RGB))
ax[1].set_title('Blue Channel')
for a in ax:
    a.axis('off')
plt.show()
```

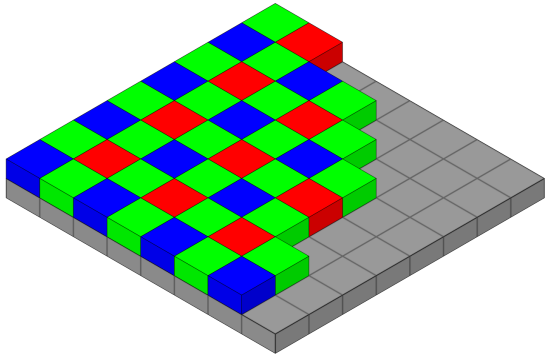


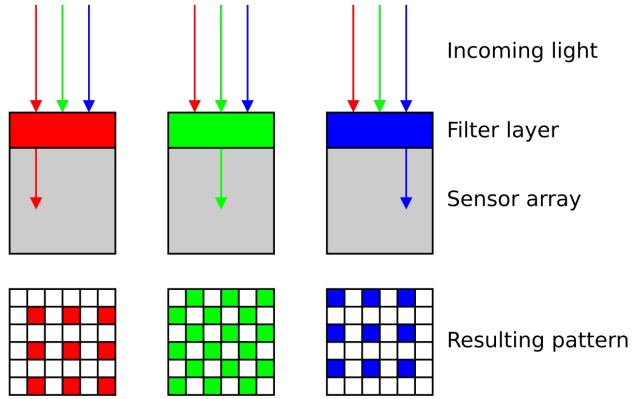
Working of a Camera



6 – CMOS sensor
it is sensitive to light not colors
so we put R/G/B filters in front of it.

Bayer arrangement of color filters



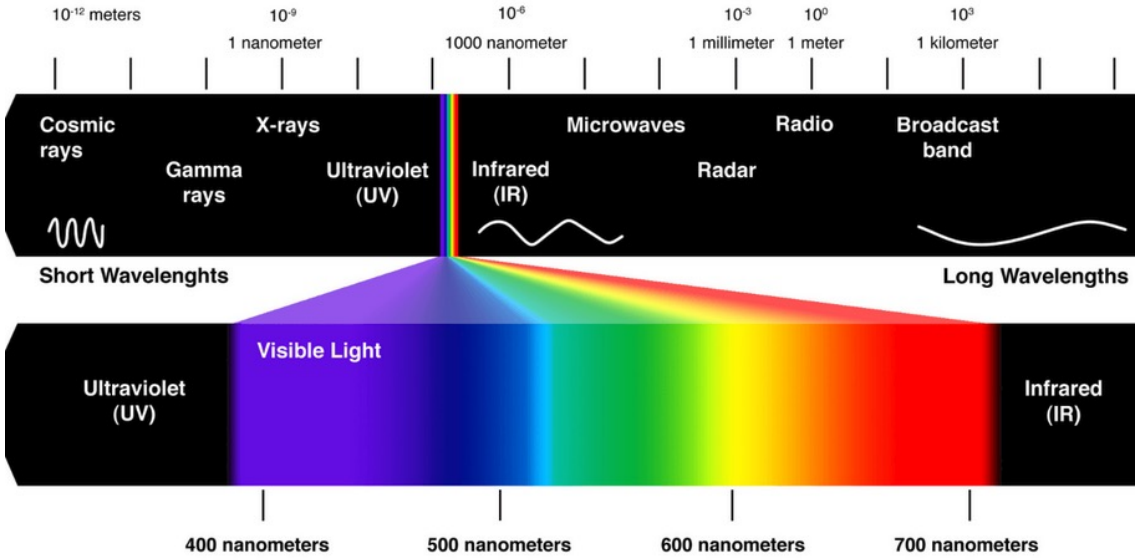


Our eyes are more sensitive to green

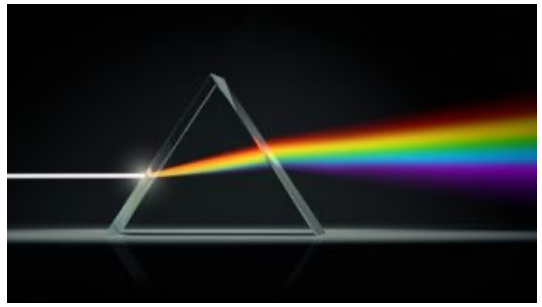
Demosaicing:
Estimation of missing components from neighboring values

https://en.wikipedia.org/wiki/Bayer_filter
https://commons.wikimedia.org/wiki/File:Cross_section_of_a_DSLR_camera.png

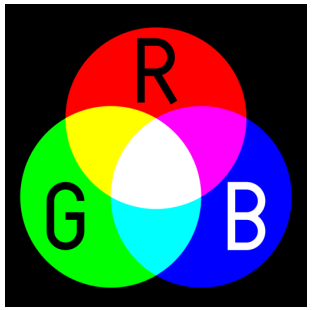
Color



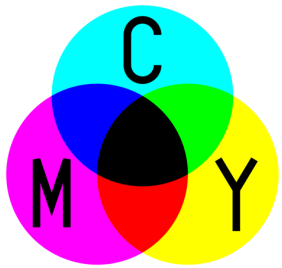
Wavelengths comprising the visible range of the electromagnetic spectrum.



Color spectrum seen by passing white light through a prism.



Additive mixing



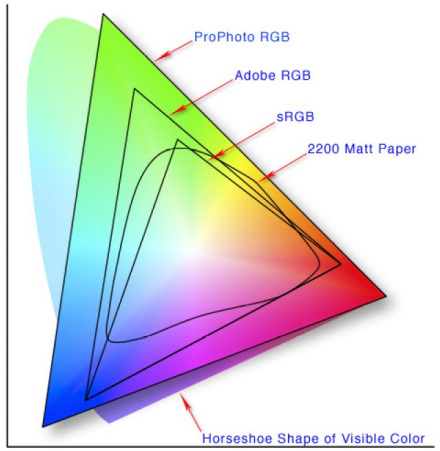
Subtractive mixing

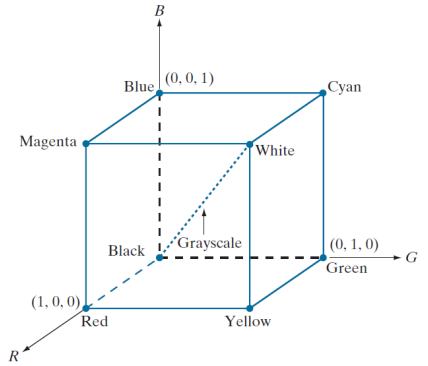
A **color model** (color space or color system) facilitates the specification of colors: (1) a coordinate system, and (2) a subspace within that system, such that each color in the model is represented by a single point contained in that subspace.

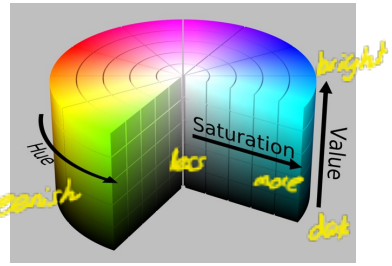
RGB: image capturing in a camera, wavelength-based

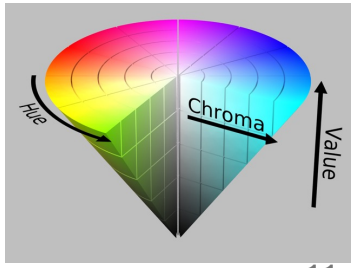
CMYK (Cyan, Magenta, Yellow, Black): for printing

HSV (Hue, Saturation, Value): how humans describe color, decouples the color and gray-scale information









Billmeyer and Saltzman's Principles of Color Technology
Gonzalez and Woods, chapter 6, https://en.wikipedia.org/wiki/Color_space
https://en.wikipedia.org/wiki/HSL_and_HSV, https://www.livescience.com/50678-visible-light.html
https://www.quora.com/What-is-visible-light-and-how-much-of-the-spectrum-does-it-cover-compared-to-the-others

Summary

1. A grayscale digital image is an array of 8-bit unsigned integers in [0, 255]. We call each integer a pixel.
2. Color images have three such arrays (planes), one for red, one for green, and one for blue.
3. We can manipulate an image using OpenCV function or a pair of for-loops to access each pixel (slower).

printers are in the substractive color space

To get rid of aliasing
- Sample more often
- remove freq higher than half the sampling freq using gaussian filters.

Intensity Transformations

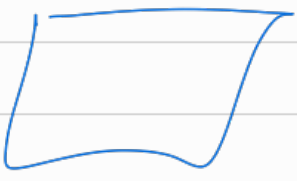
If we downsample it will cause color due to aliasing

but if do filtering first then downsamplers will not be that bad

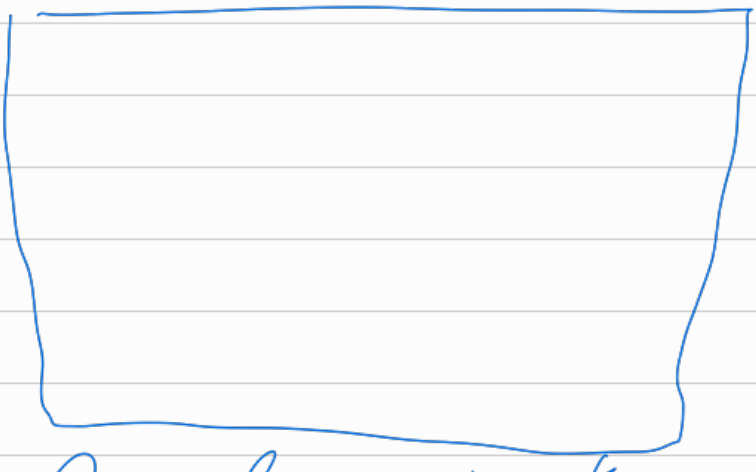
---

Try bilinear interpolation

original



Result zoom in



Run keepr in the result image.

high freq in image

0 255 0 255

Intensity Transformations: Introduction

t is the transform

In intensity transformations, the output value of the pixel depends only on the input values of that pixel, not it neighbors.

alternatively can use cv. LUT()

Input image: $f(\boldsymbol{x})$
Output image: $g(\boldsymbol{x})$
Intensity transform $g(\boldsymbol{x}) = T(f(\boldsymbol{x}))$

E.g., identity transform $T(.) = 1$

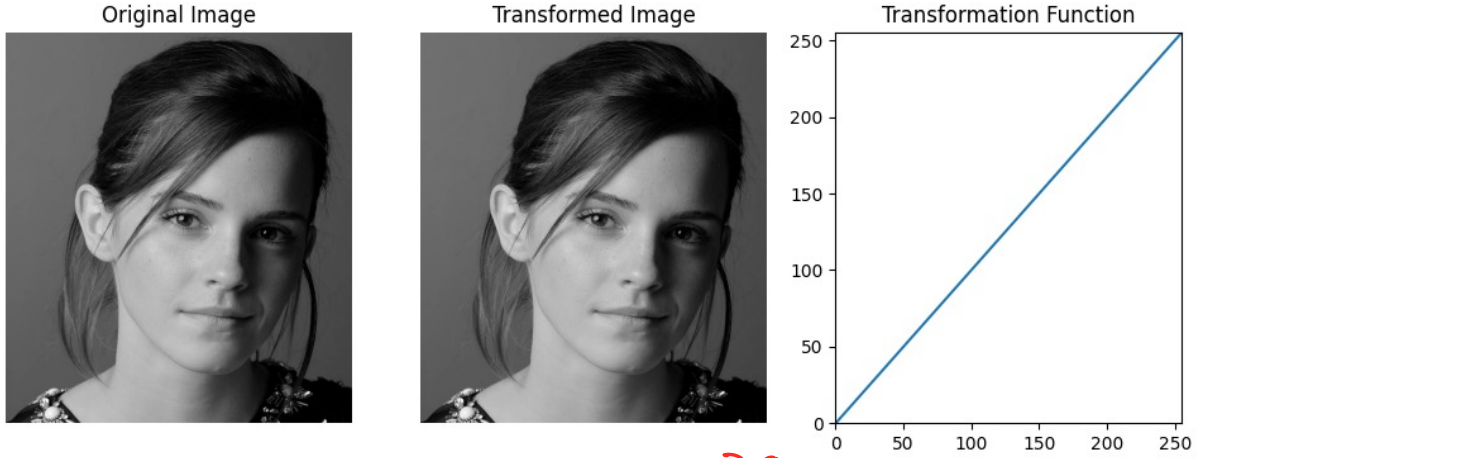
use this data type to reduce space. since we are only using 8-bit images uint8 is enough.

```
import cv2 as cv
import numpy as np
f = cv.imread('images/emma_gray.jpg',
cv.IMREAD_GRAYSCALE)
t = np.arange(256, dtype=np.uint8)
g = t[f]
```

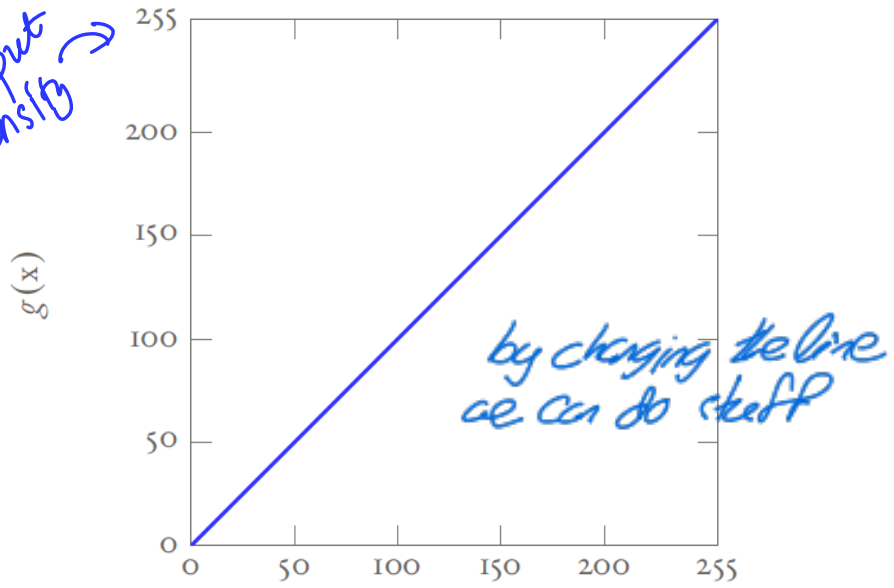
0-255

prepare the transformation

index the transform using image



output intensity



by changing the line we can do stuff

input intensity

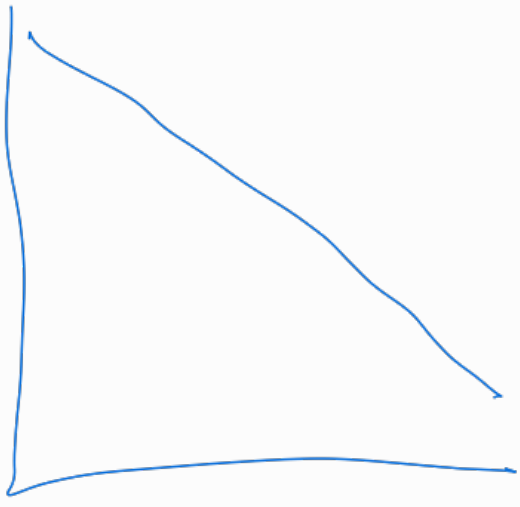
g

1. Explain the line image_transformed = cv.LUT(img_orig, transform).
2. This can be done using NumPy only as image_transformed = transform[img_orig]. Explain this operation.
3. What is $T()$ here?

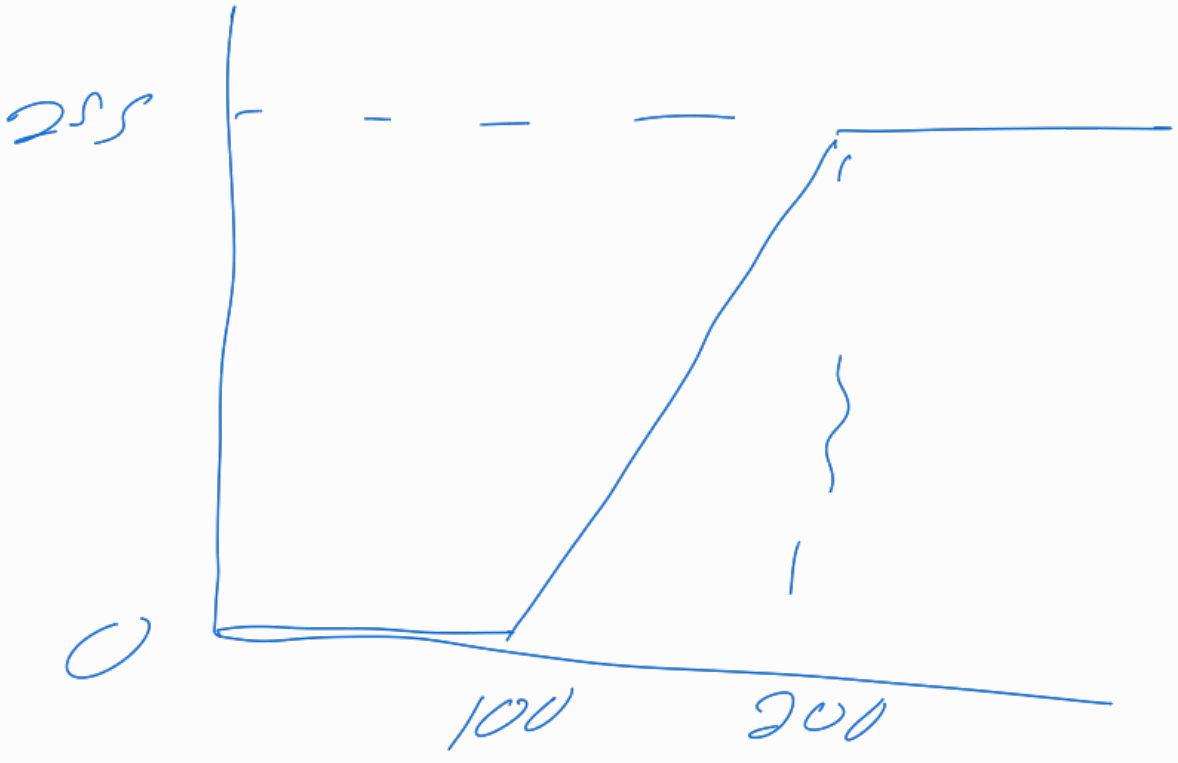
g=t[f]

To do negative transformation

t = np.arange(255, -1, -1)



Intensity windowing

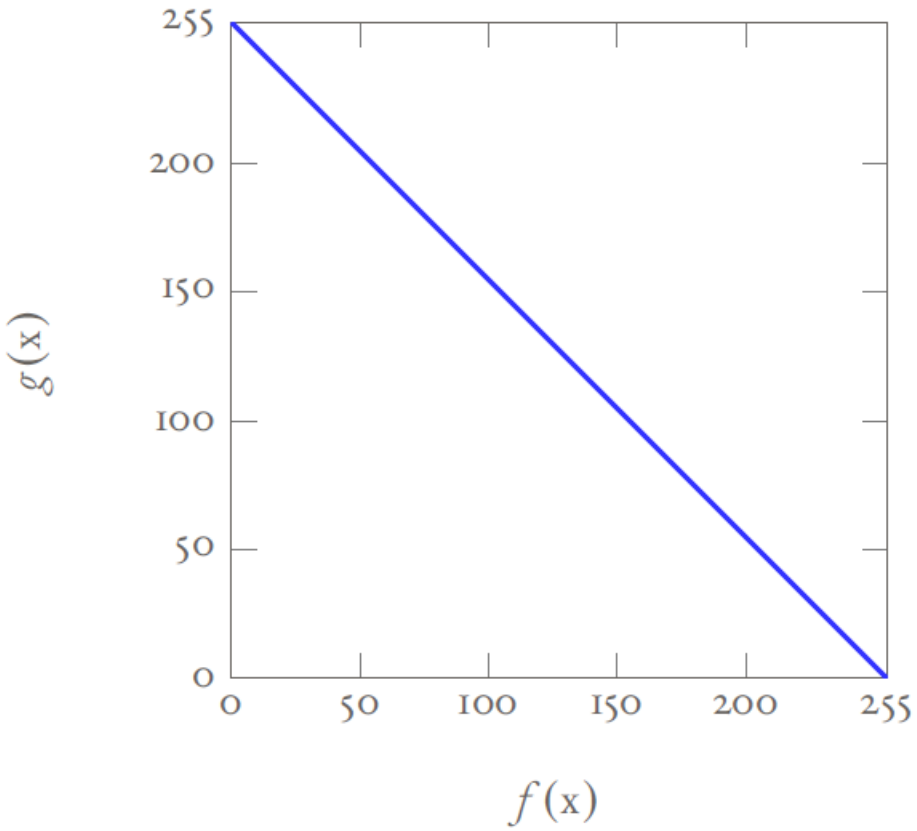


The input range 100-200 is given by a wider range

Example

In here we are doing nothing identity transform

t[25] = 25
t[[27, 25, 100]] = [27, 25, 100]
t[in] = in
so no change since the t is one to one mapping



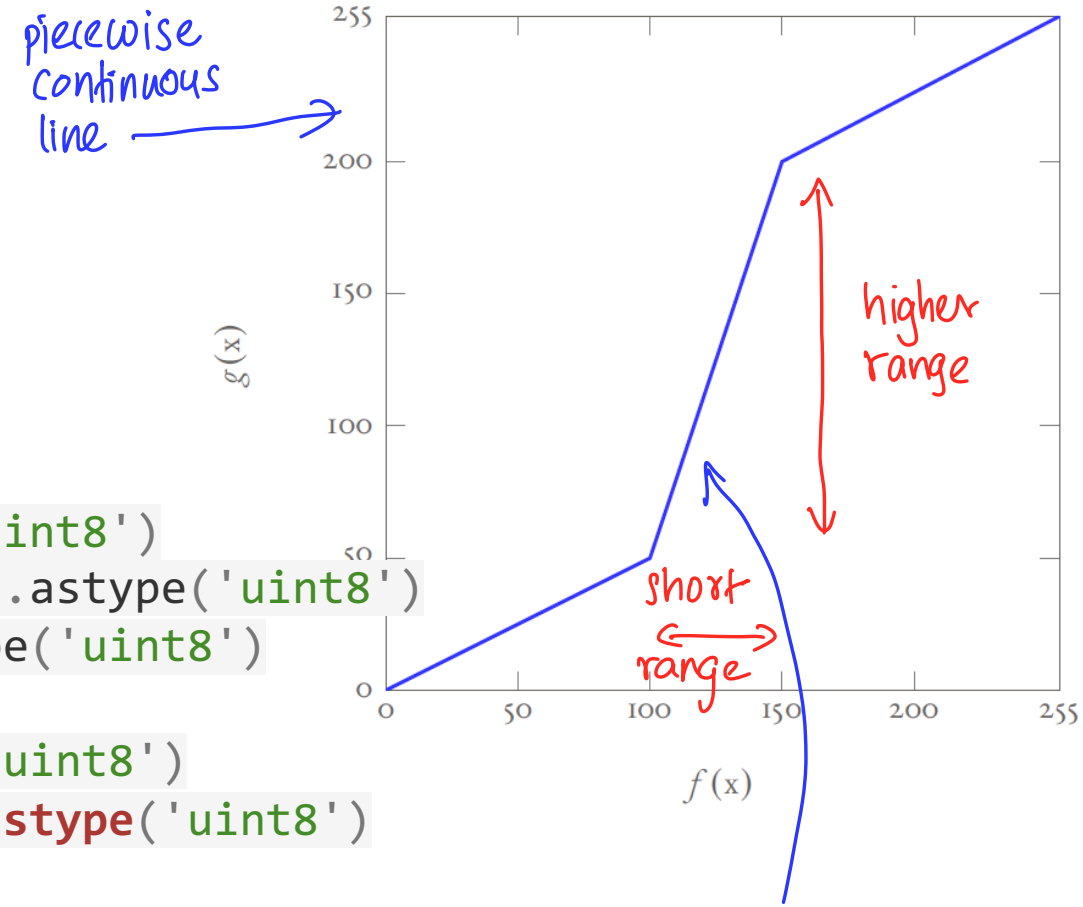
g(x) = 255 − f (x)
Implementation:

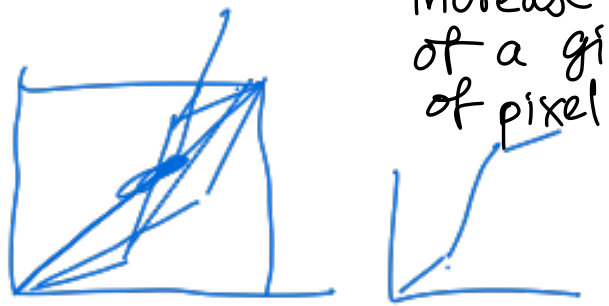
```
t = np.arange(255, -1, -1,  dtype=np.uint8)
```

negative transform
– helps in radiological images to see tumors better

Intensity Windowing

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

c = np.array([(100, 50), (150, 200)])

t1 = np.linspace(0, c[0,1], c[0,0] + 1 - 0).astype('uint8')
t2 = np.linspace(c[0,1] + 1, c[1,1], c[1,0] - c[0,0]).astype('uint8')
t3 = np.linspace(c[1,1] + 1, 255, 255 - c[1,0]).astype('uint8')

transform = np.concatenate((t1, t2), axis=0).astype('uint8')
transform = np.concatenate((transform, t3), axis=0).astype('uint8')
print(len(transform))

img_orig = cv.imread('images/katrina.jpg', cv.IMREAD_GRAYSCALE)
image_transformed = cv.LUT(img_orig, transform)
```





intensity windowing
increase the range of a given range of pixel values

Gamma Correction

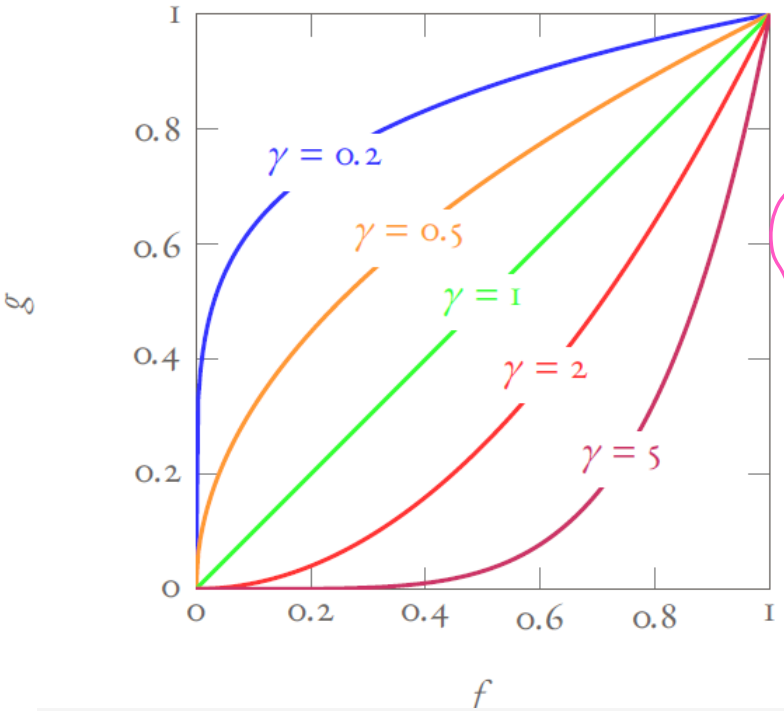
$g = f^{\gamma}, \quad f \in [0,1]$ ← image has to be normalized

Values of γ such that $0 < \gamma < 1$ map a narrow range of dark pixels to a wider range of dark pixels.

$\gamma > 0$ has the opposite effect.

$\gamma = 1$ gives the identity transform.

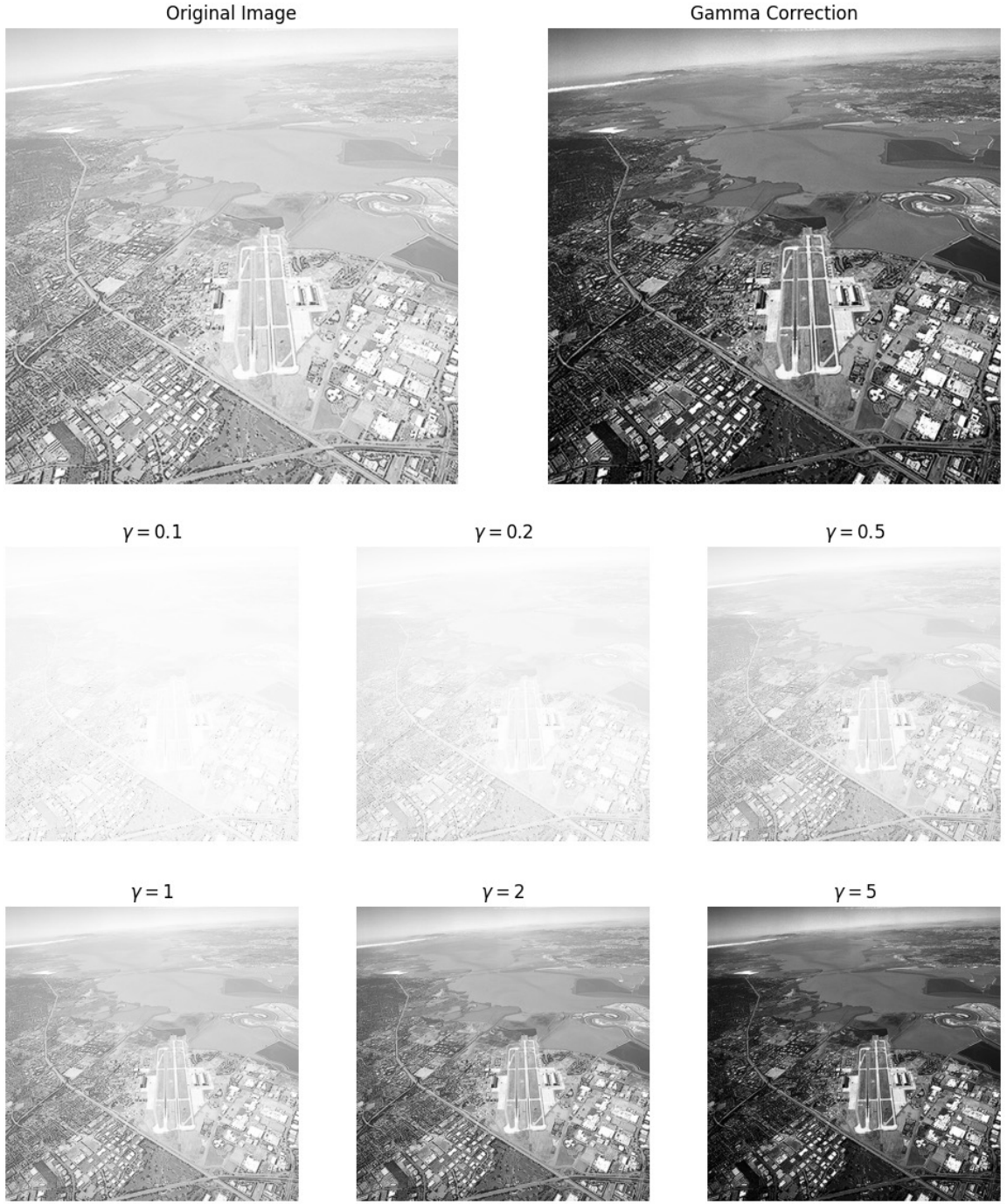
* due to dynamic range of lighting



what is the intuition?

↳ change the range of values for dark/bright pixels

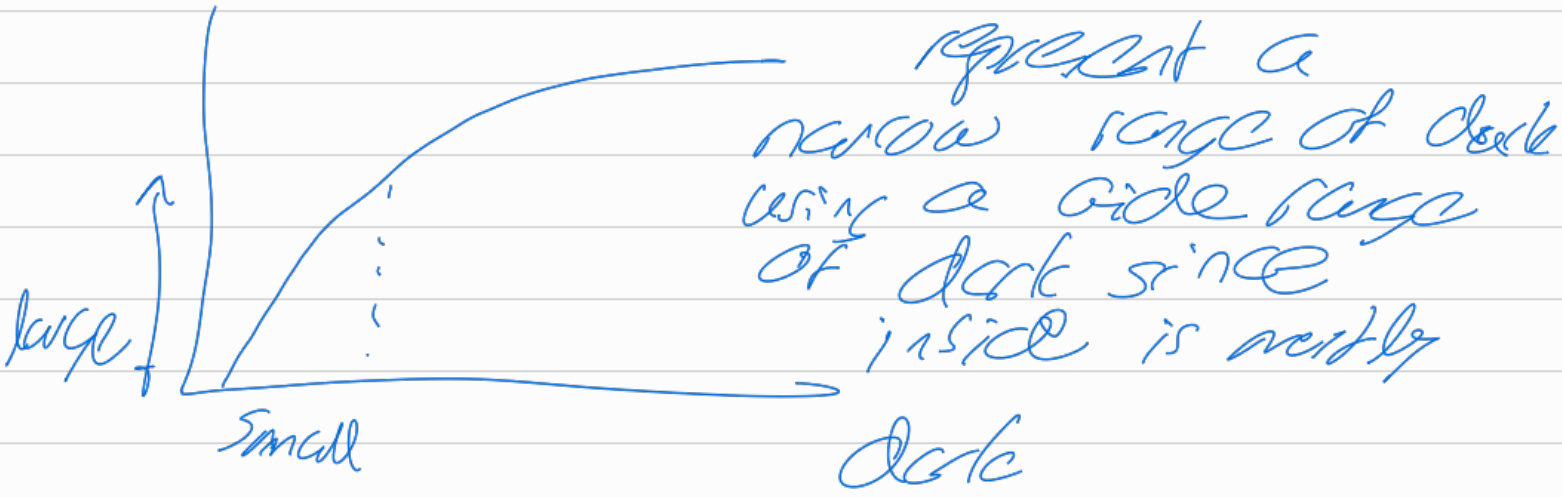
```
t = np.array([(i/255.0)**(gamma)*255 for i in
np.arange(0,256)]).astype(np.uint8)
g = cv.LUT(f, t)
```



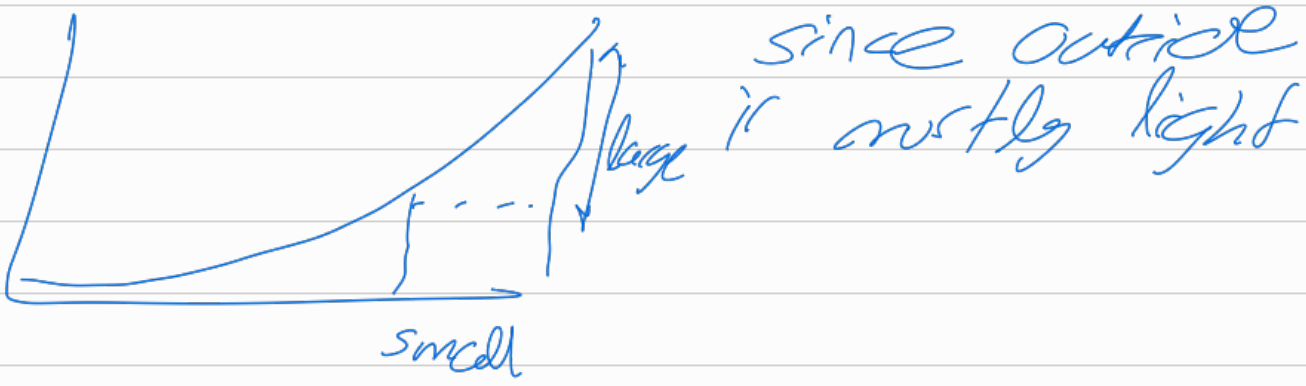
why gamma correction

inside of a room is dark compared to outside

so if we want to see inside of a room

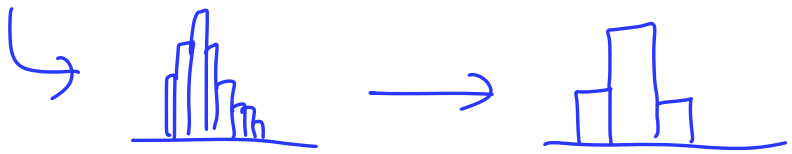


similarly to see outside

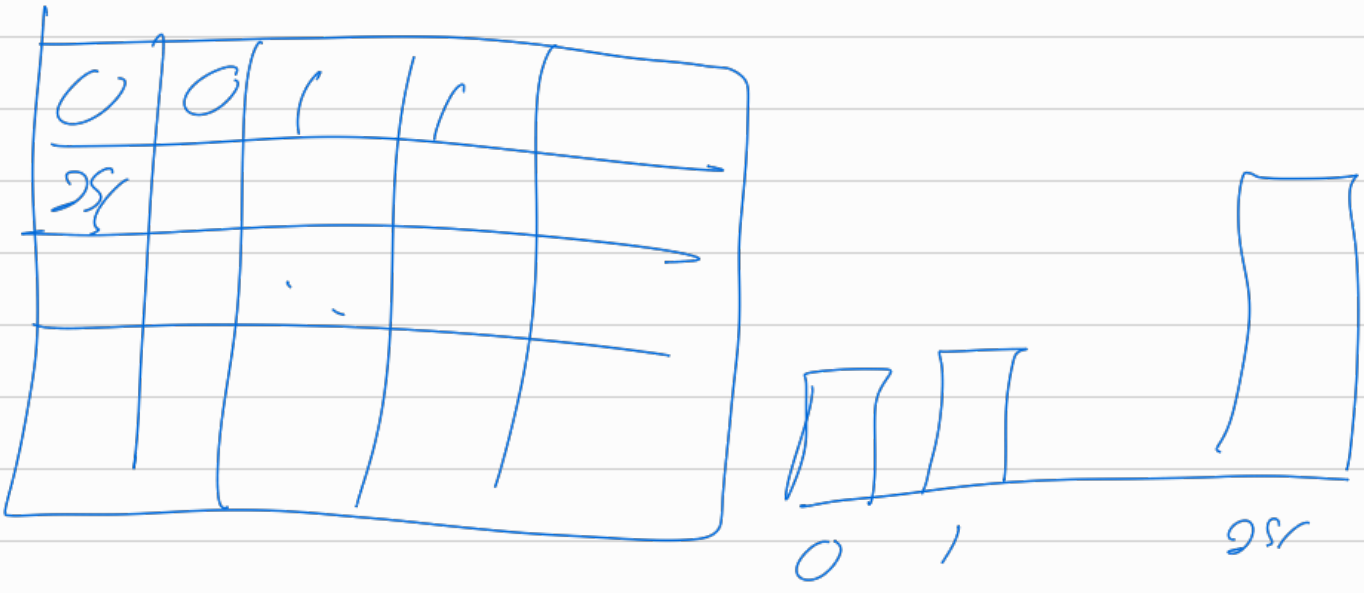


Histograms

1. We can represent the intensity distribution over the range of intensities [0, 255], using a histogram.
2. If h is the histogram of a particular image, $h(r_k)$ gives us how many pixels have the intensity r_k.
3. The histogram of a digital image with gray values in the range $[0, L-1]$ is a discrete function $h(r_k) = n_k$ where r_k is the kth gray level and n_k is the number of pixels having gray level r_k.
4. We can normalize the histogram by dividing by the total number of pixels n. Then we have an estimate of the probability of occurrence of level r_k , i.e., $p(r_k) = n_k/n$. → probability mass function
5. The histogram that we described above has L bins. We can construct a coarser histogram by selecting a smaller number of bins than L. Then several adjacent values of k will be counted for a bin.



Histogram



By dividing by total we get pmf

Exercise

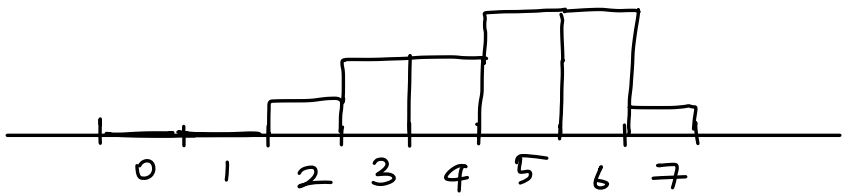
3-bit image

The figure shows a 3×4 image. The range of intensities that this image has is [0, 7]. Compute its histogram.

6	5	5	3
7	6	6	4
2	3	5	4

$h(0) = 0$
$h(1) = 0$
$h(2) = 1$
$h(3) = 2$
$h(4) = 2$
$h(5) = 3$
$h(6) = 3$
$h(7) = 1$

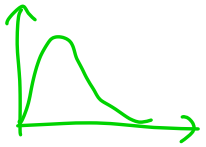
$\sum_i h(i) = 3 \times 4 = 12$
total no. of pixels



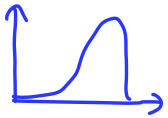
histogram

* find algorithms for calculating histograms.

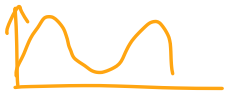
Image Properties though the Histrogram



1. If the image is dark histogram will have many values in the left region, that correspond to dark pixels.



2. If the image is bright histogram will have many values in the right region, that correspond to bright pixels.



eg: fingerprint images

3. If a significant number of pixels are dark and a significant number of pixels a bright, the histogram will have two modes, one in the left region and the other in the right region.
4. A flat histogram signifies that the image has a uniform distribution of all intensities. Then, the contrast is high, and image will look vibrant.

photographers like this

photographer's desire

Histogram Equalization

L is the bin number or more accurately the intensity

1. Photographers like to shoot pictures with a flat histogram, as such pictures are vibrant.
2. Histogram equalization is a gray-level transformation that results in an image with a more or less flat histogram.
3. We can take and image and make its histogram flat by using the operation called histogram equalization.

$$s = T(r) = (L-1)\int_0^r p_r(w)dw$$

$$s = T(k) = \frac{L-1}{MN}\sum_{j=0}^{k} n_j, \qquad k = 0,..,L-1$$

new intensity

CDF

intensity

PDF → CDF

if $k = 100$ then we add intensities upto 100

Example:
Suppose that a 3-bit image ($L = 8$) of size 64×64 pixels ($MN = 4096$) has the intensity distribution shown in Table 4, where the intensity levels are in the range $[0, L-1] = [0, 7]$. Carry out histogram equalization.

MN = 4096 (rows × cols)
L : no of colours
here there are 8 colours

PMF

cumulative density function

r_k	n_k	$p_r(r_k) = n_k/MN$	$\sum_{j=0}^{k} n_j$	$\frac{(L-1)}{MN}\sum_{j=0}^{k} n_j$	Rounded
$r_0 = 0$	790	0.19	790	1.35	1
$r_1 = 1$	1023	0.25	1813	3.098	3
$r_2 = 2$	850	0.21	2663	4.55	5
$r_3 = 3$	656	0.16			
$r_4 = 4$	329	0.08			
$r_5 = 5$	245	0.06			
$r_6 = 6$	122	0.03			
$r_7 = 7$	81	0.02	4096	7	7

So the transformation is
Total no. of pixels up to that intensity divided by Total pixels and multiplied by (L-1)

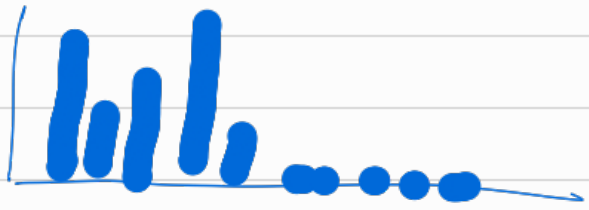
r_k	n_k	$p_r(r_k) = n_k/MN$	$\sum_{j=0}^{k} n_j$	$\frac{(L-1)}{MN}\sum_{j=0}^{k} n_j$	Rounded
$r_0 = 0$	790	0.19	790		
$r_1 = 1$	1023	0.25			
$r_2 = 2$	850	0.21			
$r_3 = 3$	656	0.16			
$r_4 = 4$	329	0.08			
$r_5 = 5$	245	0.06			
$r_6 = 6$	122	0.03			
$r_7 = 7$	81	0.02			

r_k	n_k	$p_r(r_k) = n_k/MN$	$\sum_{j=0}^{k} n_j$	$\frac{(L-1)}{MN}\sum_{j=0}^{k} n_j$	Rounded
$r_0 = 0$	790	0.19	790		
$r_1 = 1$	1023	0.25	1813		
$r_2 = 2$	850	0.21	2663		
$r_3 = 3$	656	0.16	3319		
$r_4 = 4$	329	0.08	3648		
$r_5 = 5$	245	0.06	3893		
$r_6 = 6$	122	0.03	4015		
$r_7 = 7$	81	0.02	4096		

r_k	n_k	$p_r(r_k) = n_k/MN$	$\sum_{j=0}^{k} n_j$	$\frac{(L-1)}{MN}\sum_{j=0}^{k} n_j$	Rounded
$r_0 = 0$	790	0.19	790	1.350	1
$r_1 = 1$	1023	0.25	1813	3.098	3
$r_2 = 2$	850	0.21	2663	4.551	5
$r_3 = 3$	656	0.16	3319	5.672	6
$r_4 = 4$	329	0.08	3648	6.234	6
$r_5 = 5$	245	0.06	3893	6.653	7
$r_6 = 6$	122	0.03	4015	6.862	7
$r_7 = 7$	81	0.02	4096	7.000	7

Explanation:

First we get the original histogram.



Then get the cumulative sum and apply the algorithm

Then we get for those in bin (i)

0 → 1
1 → 3
⋮
6 → 7
7 → 7

now $t = [1\ 3\ 5\ 6\ 6\ 7\ 7\ 7]$

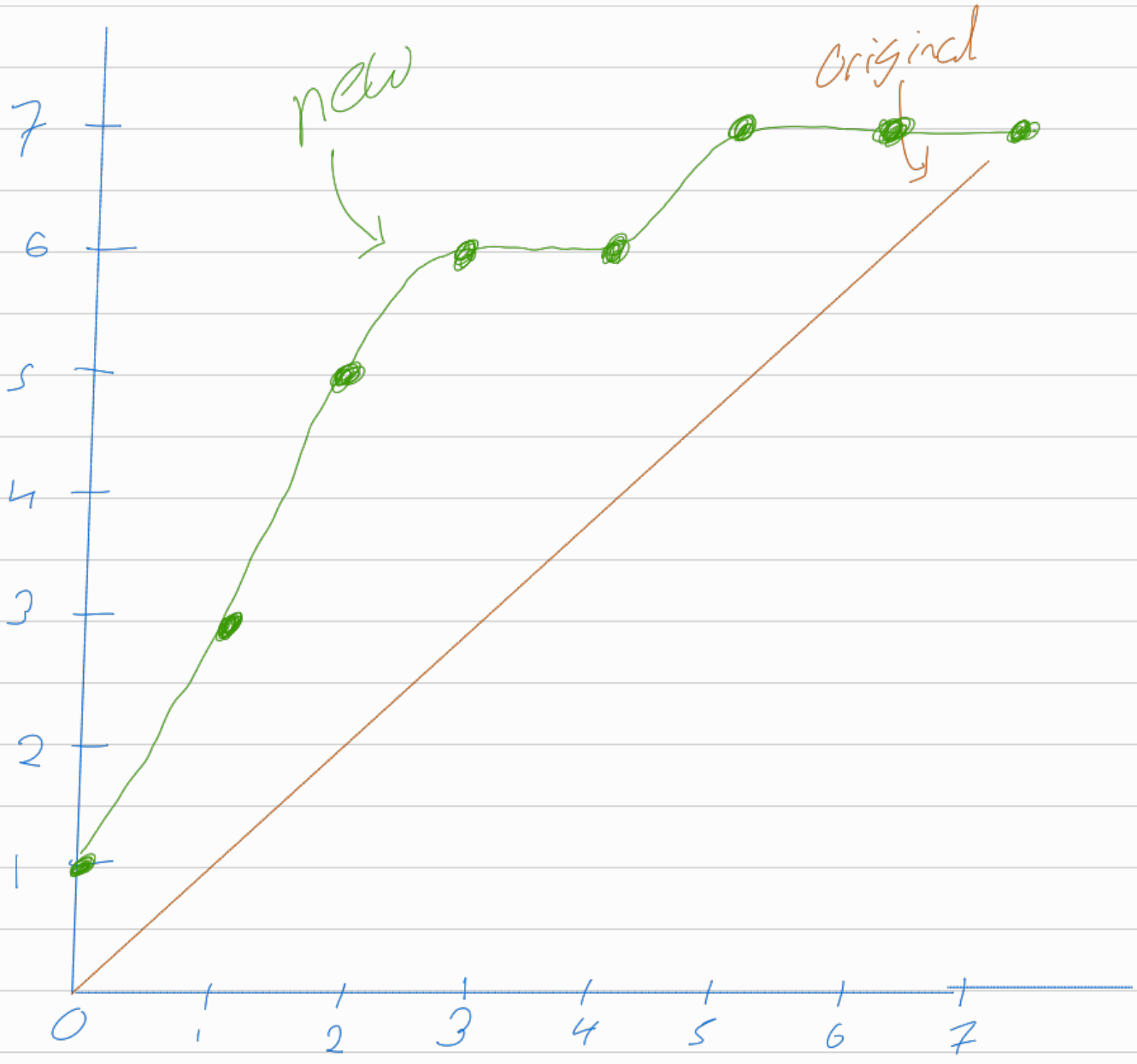
t: 1 3 5 6 6 7 7 7
index: 0 1 2 3 4 5 6 7

This is the new lookup table
now when we say $g = t[f]$ the original image changes accordingly. we can see that multiple bins get collected to a single bin

$3, 4 \rightarrow 6$

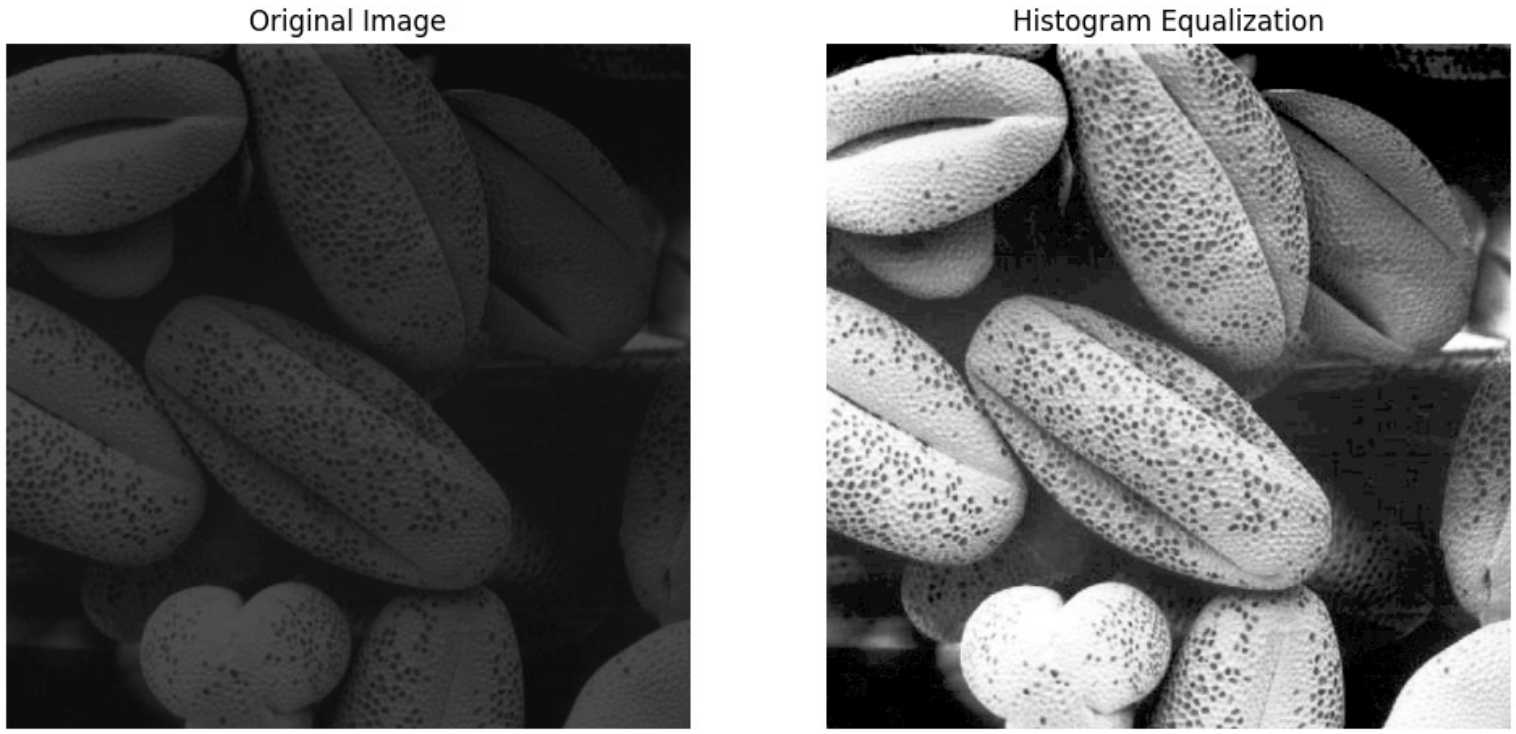
$5, 6, 7 \rightarrow 7$

} so some middle bins become empty, as see


new
original
7
6
5
4
3
2
1
0
1
2
3
4
5
6
7

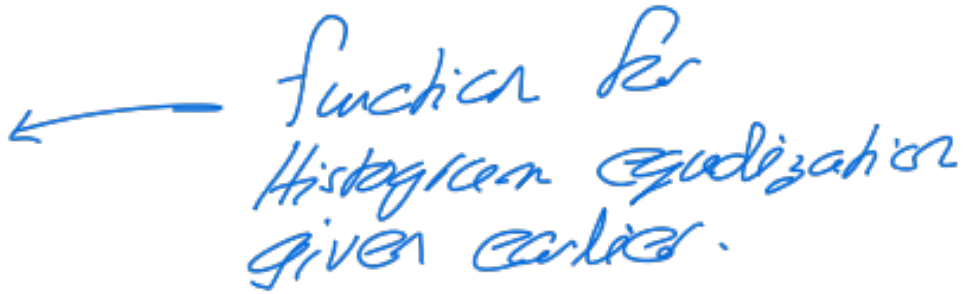
Histogram Equalization Using OpenCV

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

f = cv.imread('images/shells.tif', cv.IMREAD_GRAYSCALE)
g = cv.equalizeHist(f)
```



Histogram Equalization Using the Formula

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

f = cv.imread('images/shells.tif', cv.IMREAD_GRAYSCALE)

t = np.array([(L-1)/(M*N)*cdf[k] for k in range(256)],
dtype=np.uint8)
g = t[f]

g = cv.equalizeHist(f)
fig, ax = plt.subplots(1, 2, figsize=(12, 8))
ax[0].imshow(f, cmap='gray', vmin=0, vmax=255)
ax[0].set_title('Original Image')
ax[0].axis('off')
ax[1].imshow(g, cmap='gray', vmin=0, vmax=255)
ax[1].set_title('Histogram Equalization')
ax[1].axis('off')
plt.show()
```



Summary

1. Basics: digital image, color images, creating an image in Python, displaying images, increasing brightness, image planes
2. Intensity transformations:

$$g = t[f]$$

    1. Identity, negative, intensity windowing
    2. Gamma correction
    3. Histograms, histogram equalization

check more on → histogram matching